STRUCTURED TECHNICAL REPORT

Home Lab Configuration

Planning, Installing, Troubleshooting, and Validating a Personal Security Lab

VirtualBox | Ubuntu | Wazuh | Sysmon | Splunk | Nmap | Wireshark

PREPARED BY	Jackson Blase

REPORT DATE	November 10, 2026
PROJECT PERIOD	September 2026

Table of Contents

Bottom Line Up Front (BLUF) **4**

1. Purpose, Scope, and Plan **4**
- 1.1 Learning Objective 4
- 1.2 Scope and Boundaries 4
- 1.3 Build Sequence 5

2. Architecture and Application Roles **5**
- 2.1 Environment 5
- 2.2 What Each Application Does 6

3. Foundation Software **7**
- 3.1 Acquisition, Integrity Checks, and Versions 7
- 3.2 Wireshark and Npcap 8
- 3.3 Sysmon Installation and Configuration 8
- 3.4 Splunk Installation and First Searches 9

4. VirtualBox and Ubuntu Server **9**
- 4.1 VirtualBox and the VM Definition 9
- 4.2 Ubuntu Installer Decisions 10
- 4.3 Host-Only Network and Netplan 10
- 4.4 SSH Administration 11

5. Wazuh Server and Windows Agent **11**
- 5.1 All-in-One Installation 11
- 5.2 Stabilization and Credential Handling 12
- 5.3 Windows Agent Enrollment 12
- 5.4 Add the Sysmon Channel 13

6. Splunk Windows and Sysmon Integration **13**
- 6.1 Standard Windows Logs 13
- 6.2 Sysmon Input File 13

7. Protection, Host Settings, and Recovery Points **14**
- 7.1 Endpoint Protection 14
- 7.2 Native Windows Hardening 14
- 7.3 Pre-Investigation Snapshots 14

8. Network Analysis and Validation **15**
- 8.1 Nmap Installation and PATH Repair 15
- 8.2 Host Discovery 15
- 8.3 Targeted SYN Scan 15
- 8.4 Service-Version Probing 16
- 8.5 Validate the Owning Processes Inside Ubuntu 16
- 8.6 Useful Display Filters 17

9. Troubleshooting Record **17**
- 9.1 Method 17
- 9.2 Other Problems and Fixes 18

10. Validation and Readiness **18**

11. Lessons Learned and Future Plans **18**
- 11.1 Lessons Learned 18

11.2 Accepted Trade-Offs 19
11.3 Remaining Plans 19
12. Conclusion and Handoff 20
Appendix A. Evidence Integrity and Provenance 21
Appendix B. Sources and References 21
Glossary 22

Bottom Line Up Front (BLUF)

A working personal security lab was assembled and validated before the first SOC investigation. A Windows 11 desktop supplies endpoint events and runs local Splunk searches. An Ubuntu Server virtual machine hosts the Wazuh manager, indexer, dashboard, and Filebeat. Sysmon provides detailed endpoint telemetry, the Wazuh agent forwards selected Windows events, and Wireshark and Nmap support packet analysis and controlled network discovery.

The configuration and validation work established two independent search paths for Windows events, working host-to-VM administration, known service exposure, recovery snapshots, and a repeatable troubleshooting method. This report covers setup only, from planning through pre-case validation, and closes with lessons learned, accepted trade-offs, and open plans. The first investigation is documented separately in *SOC Investigation #1*.

> **Source Boundary.** This report documents the home lab as it was planned, built, and validated. Saved scripts, the Sysmon XML configuration, the VirtualBox machine definition, the network-analysis summary, and three original packet captures support it. Packet counts and SHA-256 values were independently rechecked on November 10, 2026, and service states are reported as observed during setup, not as current readings. [L1–L4]

1. Purpose, Scope, and Plan

1.1 Learning Objective

The objective was to turn networking and security knowledge into practical skill: install and administer a monitoring platform, generate and find known events, inspect network traffic, troubleshoot failures, and document the results. The lab provides an environment for later alert triage and investigation; it does not represent employment or a production SOC.

The initial plan combined a small defensive lab, guided SOC practice, and written case studies. The immediate technical priority was a reliable evidence pipeline. Optional enterprise tools, additional VMs, ticketing integrations, and response automation were deferred until core collection and search worked; Section 11.3 lists them.

1.2 Scope and Boundaries

The Windows host and the Ubuntu VM are owned and controlled by the author. Active discovery and scans targeted only the internal Wazuh VM at 192.168.56.10. The monitored Windows endpoint uses the Wazuh agent name SOC-Windows-01 and the hostname WORKSTATION-01.

The VM has two network adapters. The host-only adapter keeps host-to-server lab traffic on a private virtual link, and the NAT adapter provides outbound internet access for packages and updates. The lab is therefore not air-gapped, and the physical Windows host is also a personal workstation.

1.3 Build Sequence

Stage	Work Performed	Completion Evidence
Foundation	Obtain installers; verify publishers and checksums; install packet-capture and endpoint tools	Historical installer record and saved foundation files
Virtualization	Install VirtualBox and create the Ubuntu Server VM	Saved VirtualBox definition and three pre-case snapshots
Networking	NAT for updates; static host-only server address; SSH	Recorded Netplan file, reachability tests, server login, and later captures
Monitoring Platform	Install Wazuh all-in-one; stabilize services; rotate the exposed initial admin password	Historical service checks and snapshot description
Endpoint Integration	Deploy the Windows agent; add the Sysmon channel	Agent Active, accepted event-channel configuration, and test events
Independent Analytics	Configure local Windows and Sysmon inputs in Splunk	`btool` configuration check and `whoami` event search
Protection and Recovery	Review endpoint protection and native hardening; take a VM baseline snapshot	Historical checks and snapshot metadata
Network Validation	Host discovery, SYN scan, service probes, and listener verification	Three original captures and server-side process notes
Readiness	Confirm evidence generation, search, access, and boundaries	Ready to begin the first SOC investigation

2. Architecture and Application Roles

2.1 Environment

Physical Host	Windows 11 Pro, hostname WORKSTATION-01, AMD Ryzen 7 9700X, 64 GB RAM. The purchase and assembly record and the November hardware inventory corroborate the processor correction documented in *PC Build*. [L10]
Virtualization	Oracle VirtualBox 7.2.18, as recorded in the VM definition
Virtual Machine	Wazuh-Server running Ubuntu Server 24.04.5 LTS; 4 vCPUs, 8,192 MB RAM, and an 80 GB dynamically allocated VDI
Host-Only Link	Windows 192.168.56.1/24; Ubuntu `enp0s8` 192.168.56.10/24
VM Internet Path	`enp0s3`, DHCP through NAT; recorded address 10.0.2.15/24
Wazuh	4.14 all-in-one deployment; Windows agent package 4.14.7-1; agent SOC-Windows-01, ID 001
Splunk	Splunk Enterprise 10.4.3, hosted locally; Splunk Web at `localhost:8000`
Telemetry	Sysmon 15.22; channel `Microsoft-Windows-Sysmon/Operational`

Packet and Discovery Tools	Wireshark 4.6.8 during setup (4.6.9 by the investigation); Npcap 1.88; Nmap 7.991

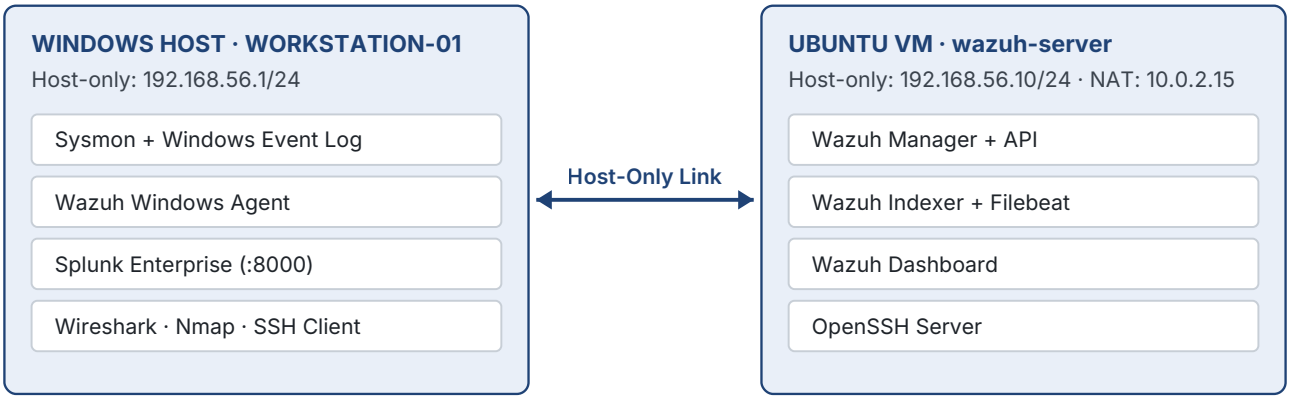


Figure 1. *Lab architecture, drawn from the lab configuration and the saved VM definition. Private host-only management and telemetry traffic is separate from the VM's NAT update path. [L3]*

2.2 What Each Application Does

Component	Purpose	How It Fits
VirtualBox	Provides virtual CPUs, memory, disks, and network adapters	Hosts the Linux monitoring server separately from Windows
Ubuntu Server	Linux operating system and service management	Runs the Wazuh components and provides SSH administration
Sysmon	Records detailed process, network, file, registry, DNS, and related events according to its configuration	Produces endpoint evidence; it does not judge whether activity is malicious
Windows Event Viewer	Displays local Windows event records	Confirms the source events before downstream copies are relied on
Wazuh Agent	Reads configured event channels and forwards events	Connects the Windows endpoint to the manager over TCP 1514
Wazuh Manager	Decodes incoming events and applies detection rules	Produces alerts with rule IDs and severity levels
Filebeat, Indexer, Dashboard	Ship, store, search, and display Wazuh alert data	Provide the analyst's alert-review interface
Splunk Enterprise	Independently collects and indexes local Windows logs	Uses SPL for searches, grouping, raw-event review, and timelines
Wireshark and Npcap	Capture packets on a selected interface and decode protocols	Provide network evidence independent of endpoint logs
Nmap	Performs authorized host discovery, port scans, and service probes	Tests the server's expected exposure and generates traffic to analyze

Component	Purpose	How It Fits
PowerShell and SSH	Local checks and remote administration	Inspect services, timestamps, files, logs, listeners, and Linux state
Endpoint Protection	Malwarebytes plus built-in Windows security features	Protects the host and offers a separately evaluated evidence source

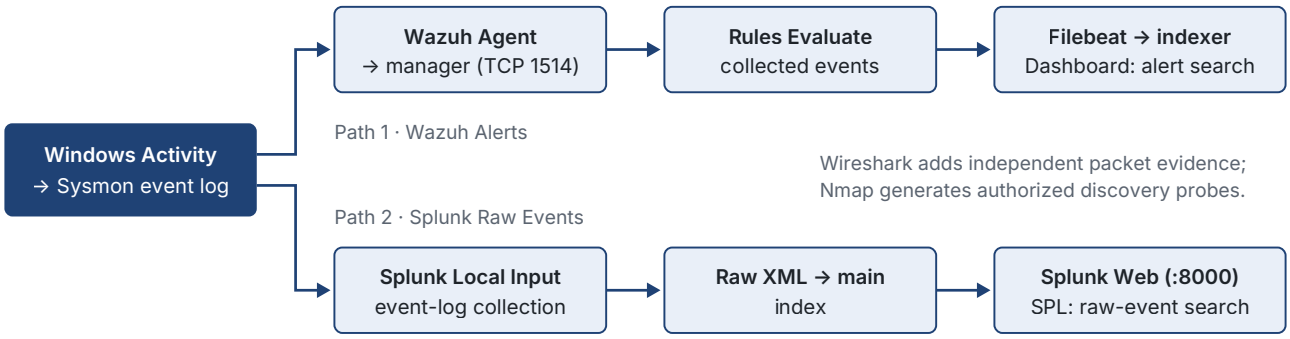


Figure 2. *The two event pipelines. Wazuh alert searches and Splunk raw-event searches are different views with different coverage; a missing alert does not establish that the underlying activity was absent. [L1, L5]*

3. Foundation Software

3.1 Acquisition, Integrity Checks, and Versions

Each installer came from its official vendor source and was checked by signature or checksum where available. A downloaded file or prepared script is not proof that it was executed; the final configuration and later tests establish which deployment was actually used.

Package	Recorded Version	Recorded Acquisition and Verification
Wireshark	4.6.8 x64 (initial)	Valid Wireshark Foundation Authenticode signature
Sysmon	15.22	Microsoft-signed executable; ZIP hash recorded
Splunk Enterprise	10.4.3	Splunk LLC signature; SHA-512 compared with the official checksum
VirtualBox	7.2.18	Oracle signature; SHA-256 compared with SHA256SUMS; Extension Pack not installed
Ubuntu Server ISO	24.04.5 LTS	4,080,486,400 bytes; SHA-256 compared with Canonical's checksum
Wazuh	4.14; Windows agent 4.14.7-1	Official packages.wazuh.com host and the dashboard-generated deployment command
Nmap	7.991	Official Windows installer; existing Npcap installation retained

The initial plan considered hosting Ubuntu under Windows Subsystem for Linux (WSL). The final design moved the Wazuh server to a dedicated Ubuntu VM in VirtualBox, which gives the server its own virtual hardware, network adapters, and snapshots. The foundation script saved from that early stage still references WSL; it records how the plan evolved, not the deployed design.

Downloading the Ubuntu ISO took several attempts. The browser download repeatedly stalled, and a single-stream `curl` attempt failed with a Windows TLS credential error. A resumable multi-part transfer completed the download, and the ISO was used only after its checksum matched Canonical's published value.

3.2 Wireshark and Npcap

1. Run the verified Wireshark installer and install Npcap, the required capture driver.
2. Keep the main Wireshark and TShark components. The later update kept Etwdump; the specialized remote and random-packet capture interfaces were unnecessary for this lab.
3. Select **Ethernet 2**, the VirtualBox Host-Only Ethernet Adapter at 192.168.56.1. The physical home-network adapter carries a different path.
4. Start a capture and confirm packets are arriving before generating test traffic.
5. Use display filters to isolate the relevant protocols or target, and save the original capture as PCAPNG for later analysis.

A capture filter changes what is recorded; a display filter changes only what is shown from the recorded packets. The lab used display filters, so unrelated Wazuh agent traffic remained in the saved files. As a result, each file's total size and packet count are larger than the filtered scan evidence.

3.3 Sysmon Installation and Configuration

The 64-bit Sysmon executable and the prepared XML configuration were placed in the same folder. From an Administrator PowerShell session, the service and driver were installed with that configuration:

```
& "$env:USERPROFILE\Downloads\Sysmon\Sysmon64.exe" -i `
  "$env:USERPROFILE\Downloads\Sysmon\sysmon-lab-config.xml"
```

The call operator `&` runs the quoted executable. The `-i` switch installs Sysmon, and the XML file that follows defines its event filters. The configuration's schema version 4.82 was accepted by the newer 4.91 engine, and Sysmon64 and SysmonDrv started.

The saved configuration records SHA256 and IMPHASH hashes and collects process creation broadly (excluding conhost), network connections (excluding svchost), file creation in Downloads, Temp, and Startup, selected registry autorun changes, DNS queries, named pipes, process tampering, and file deletions. Some noisy categories were intentionally left out. This is a chosen telemetry policy, not a record of every Windows action.

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5 |
  Select-Object TimeCreated, Id, ProviderName |
  Format-Table -AutoSize
```

This command reads recent records without changing them. Recent Event ID 3 records confirmed that the sensor and channel were working. In Event Viewer, the local log is under *Applications and Services Logs > Microsoft > Windows > Sysmon > Operational*. [L1, L6]

3.4 Splunk Installation and First Searches

The verified Splunk Enterprise 10.4.3 MSI installed to `C:\Program Files\Splunk`. Setup used the installer's virtual service-account option and a separate Splunk administrator login. Splunk Web opened at `http://localhost:8000` in the Search & Reporting app. The first searches explored Splunk's own diagnostic data:

```
index=_internal
| head 20

index=_internal
| stats count by sourcetype
| sort - count
```

The first search returned 20 events; the grouped search summarized 5,465 events across 24 sourcetypes. These searches introduced the default fields `index` (where events are stored), `host` (the producing system), `source` (the specific log), `sourcetype` (the data format), `_time` (the event timestamp), and `_raw` (the original event text).

An early `index=*` search found no user data because no inputs had been added yet. Splunk's internal data lives in underscore-prefixed indexes, which `index=*` does not match, so the empty result did not mean Splunk was broken. A search for calculated-field warnings also generated its own CalcFieldProcessor diagnostics; the resulting volume was a search-configuration issue, not a threat finding.

4. VirtualBox and Ubuntu Server

4.1 VirtualBox and the VM Definition

VirtualBox was installed with its networking and USB components. The expected warning about a brief network interruption was accepted during installation, and the optional Python-bindings warning did not affect normal VM operation. Expert Mode was used to expose all configuration controls.

Setting	Recorded Choice	Reason or Result
VM Name	Wazuh-Server	Distinct server identity
ISO	`ubuntu-24.04.5-live-server-amd64.iso`	Supported Ubuntu LTS release at setup time
Installation	Manual; unattended installation disabled	Explicit control over the Linux account, disks, networking, and SSH
Resources	4 vCPUs, 8 GB RAM	Leaves capacity for the host and local analytics
Storage	80 GB dynamically allocated VDI, not split	Virtual disk grows with use
Firmware	EFI unchecked	VM-level choice, independent of the physical PC's firmware
Adapter 1	NAT	Outbound package and update access
Adapter 2	Host-only	Stable private link to Windows

VirtualBox initially detected the ISO as Ubuntu 24.10. The verified ISO and the installed release confirm 24.04.5, so nothing was changed just to match the automatic label. The saved machine definition independently confirms the Ubuntu24_64 OS type, four CPUs, and 8,192 MB of RAM. [L3]

4.2 Ubuntu Installer Decisions

1. Select the standard Ubuntu Server installation rather than the minimized option, and leave third-party drivers unselected.
2. Use DHCP on the NAT interface (`enp0s3`), leave the proxy blank, and let the installer validate the selected mirror.
3. Install to the 80 GB virtual disk, not a physical Windows drive. LVM was enabled; LUKS encryption was not selected for this lab.
4. Review the storage summary. The installer initially allocated only about 39 GB to the root volume, so `ubuntu-lv` was expanded from 38.996G to 77.996G within the virtual disk.
5. Set the server name to `wazuh-server` and the Linux administrator to `labadmin`; skip Ubuntu Pro.
6. Install the OpenSSH server with password authentication enabled; no SSH key was imported, and no optional snaps were selected.
7. Reboot, log in, and apply operating-system updates.

```
sudo apt update
sudo apt upgrade -y
```

The `apt update` command refreshes the package lists, and `apt upgrade -y` installs available upgrades while automatically answering yes to prompts.

4.3 Host-Only Network and Netplan

After a clean shutdown, Adapter 2 was enabled as host-only. Ubuntu detected it as `enp0s8`, but the interface stayed down until Linux was configured to use it. The host's address was 192.168.56.1/24, and the server was assigned 192.168.56.10/24. The recorded `/etc/netplan/50-cloud-init.yaml` configuration was:

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      addresses:
        - 192.168.56.10/24
```

```
sudo cat /etc/netplan/50-cloud-init.yaml
sudo netplan generate
sudo netplan apply
ip -br address
```

The `cat` command displays the saved file; `netplan generate` validates it and builds the backend configuration; `netplan apply` activates it; and `ip -br address` shows the resulting interface state. An early edit had not actually been saved: printing the file showed that it still contained only `enp0s3`. Re-saving the file, confirming its contents, validating, and applying it brought the interface up.

4.4 SSH Administration

```
Test-NetConnection 192.168.56.10 -Port 22
ssh labadmin@192.168.56.10
```

The first command tests TCP reachability without authenticating; the second opens a remote shell. On first connection, the host-key fingerprint presented to the client was compared with the server's own public host-key fingerprint before it was accepted. Typing `exit` closes the shell; it does not shut down Ubuntu.

At first, ping worked but connections to port 22 were refused. The service, its socket activation, and the UFW firewall state were checked before anything was changed:

```
sudo systemctl status ssh --no-pager
sudo systemctl status ssh.socket --no-pager
sudo ufw status verbose
sudo systemctl disable --now ssh.socket
sudo systemctl enable --now ssh.service
```

The repair switched SSH from the problematic socket-activated configuration to a continuously running service, after which TCP/22 and login both succeeded. UFW was already inactive; it was recorded as found and was not disabled as a troubleshooting shortcut. Successful ping proved network reachability, not service availability.

5. Wazuh Server and Windows Agent

5.1 All-in-One Installation

A clean Ubuntu snapshot was taken before Wazuh was installed. The recorded assisted installation used:

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

The `curl -sO` command downloads the script silently under its original filename, `bash` runs it, and `-a` selects the all-in-one installation. This is the 4.14 command used at setup; future releases may use a different package path. The deployment places the Wazuh server, indexer, and dashboard on the Ubuntu VM. [L5]

Service or Process	Function	Recorded Listening Ports
`wazuh-manager` / `wazuh-remoted`	Analyzes events and accepts agent messages	1514/TCP
`wazuh-authd`	Agent enrollment	1515/TCP
Wazuh API (`python3 wazuh_apid.py`)	Management API	55000/TCP
`wazuh-indexer` (Java)	Alert indexing and search	9200 and 9300, loopback only
`wazuh-dashboard` (Node.js)	Web interface for analysts	443/TCP
Filebeat	Ships manager alerts to the indexer	Internal connection to the indexer
OpenSSH (`sshd`)	Server administration	22/TCP

5.2 Stabilization and Credential Handling

Immediately after installation, the dashboard showed API errors and timeouts. Service and listener checks confirmed that the manager and API were present. The VM had about 5.2 GB of available memory but a one-minute load average of 18.07 on four vCPUs. The evidence pointed to heavy initialization load rather than a failed installation. The load fell over time, and an orderly shutdown and restart followed by a fresh browser session restored the dashboard.

```
sudo systemctl status wazuh-manager --no-pager
sudo ss -ltnp
sudo journalctl -u wazuh-dashboard -n 50 --no-pager
sudo tail -n 50 /var/ossec/logs/api.log
free -h
uptime
top -b -n 1 -o %CPU
```

These read-only checks show which services are running, which ports are listening, recent application errors, available memory, system load, and the top CPU consumers. The early API errors included TLS-related messages, but the recovery does not prove a single TLS root cause; the evidence as a whole supported an overloaded initialization.

The generated Wazuh administrator password had appeared in a screenshot, so it was replaced using Wazuh's password-management tool. Filebeat was restarted and verified active, and the new dashboard login worked. No password values are reproduced in this report. Reading a new password into a variable with silent input keeps it off the screen and out of shell history; however, passing it as a command argument can still briefly expose it to other local users through the process list.

```
systemctl is-active wazuh-manager wazuh-indexer wazuh-dashboard filebeat
```

The check returned four `active` lines. A second snapshot then captured the working platform before agent deployment.

5.3 Windows Agent Enrollment

1. Confirm that the settled server is listening on ports 1514 and 1515.
2. In the dashboard's agent deployment workflow, select Windows MSI, manager 192.168.56.10, agent name SOC-Windows-01, and group `default`.
3. Run the generated installation command from Administrator PowerShell, then start the Wazuh service.
4. Verify the local service state and the dashboard enrollment. The endpoint progressed from Pending to Active as ID 001, and its hardware and security-configuration inventory arrived.

```
Invoke-WebRequest -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.7-1.msi" `
  -OutFile "$env:TEMP\wazuh-agent-4.14.7-1.msi"
msiexec.exe /i "$env:TEMP\wazuh-agent-4.14.7-1.msi" /q `
  WAZUH_MANAGER="192.168.56.10" WAZUH_AGENT_GROUP="default" `
  WAZUH_AGENT_NAME="SOC-Windows-01"
NET START Wazuh
Get-Service -Name WazuhSvc
```

The download is a separate step from installation. The `/i` switch installs the MSI and `/q` suppresses the setup wizard, so no output is expected. The `WAZUH_MANAGER` property specifies the server and `WAZUH_AGENT_NAME` identifies the endpoint. `Get-Service` provides a read-only local status check.

5.4 Add the Sysmon Channel

Installing Sysmon does not by itself make Wazuh collect its channel. The agent configuration file, `C:\Program Files (x86)\ossec-agent\ossec.conf`, was backed up before this block was added inside `<ossec_config>`:

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

After the agent was restarted, its log reported both a connection to 192.168.56.10:1514/TCP and analysis of the Sysmon channel. A harmless `whoami.exe` process was then run and found in Threat Hunting. The search returned 94 process-creation records in the selected time window; that count reflects all process starts in the window, not 94 executions of the single test command.

6. Splunk Windows and Sysmon Integration

6.1 Standard Windows Logs

Under *Settings > Data inputs > Local event log collection*, the Application, Security, and System logs were selected and stored in the `main` index. Other diagnostic channels were left unselected to limit unnecessary volume. Because Splunk runs on the same Windows computer, no Universal Forwarder was needed for these local inputs.

6.2 Sysmon Input File

The Sysmon channel did not appear in the graphical event-log selector, so a custom configuration app was created rather than editing Splunk's default files:

```
C:\Program Files\Splunk\etc\apps\homelab_windows\local\inputs.conf

[WinEventLog://Microsoft-Windows-Sysmon/Operational]
disabled = 0
index = main
renderXml = true
current_only = 1
```

The stanza enables the channel, sends events to `main`, requests XML rendering, and collects only new events instead of importing the entire existing log. XML preserves each event's named fields in the raw record, but it does not guarantee that every field is already extracted for searching, which normally requires a Splunk add-on or explicit extraction.

```
& "C:\Program Files\Splunk\bin\splunk.exe" btool inputs list `
  "WinEventLog://Microsoft-Windows-Sysmon/Operational" --debug
& "C:\Program Files\Splunk\bin\splunk.exe" restart
```

The `btool` utility displays the effective settings and the file each one comes from; the expected values came from `homelab_windows\local\inputs.conf`. Because `Restart-Service Splunkd` had failed, Splunk's own restart command was used. Afterward, `index=main "whoami.exe"` returned the test's Sysmon Event ID 1 record, with parent process, user, integrity level, source, and XML sourcetype, as well as a separate deletion record referencing its prefetch file.

7. Protection, Host Settings, and Recovery Points

7.1 Endpoint Protection

Malwarebytes, under an existing subscription, was kept as the host's primary real-time protection, and Microsoft Defender Antivirus was intentionally left in passive mode. The recorded status query reported `AMRunningMode=Passive` with current definitions, and Windows Security Center identified Malwarebytes as the registered antivirus product. Passive mode reflects this machine's configuration when another product is registered; it should not be generalized to every Windows edition. [L11]

The setup review covered Malwarebytes' real-time protection layers, brute-force protection, automatic updates, self-protection, archive scanning, PUP and PUM detection, its empty allow list, and a weekly Threat Scan. Advanced exploit-protection options were left at their defaults.

When Detection History was first needed, the Tamper Protection password was unavailable, and protection was not bypassed or disabled to reach it. The password was later recovered. Malwarebytes then confirmed the information that was needed, and all of its settings were verified as configured.

7.2 Native Windows Hardening

Memory Integrity and Local Security Authority protection were turned on, the Microsoft vulnerable driver blocklist was enforced, and Kernel-mode Hardware-enforced Stack Protection was enabled, followed by a restart. A subsequent lab check showed the Wazuh agent Active, Splunk usable, and Malwarebytes protection active.

7.3 Pre-Investigation Snapshots

The saved VirtualBox definition independently retains these three pre-case snapshot names and UTC creation times:

Snapshot	UTC Timestamp	State Captured
Clean Ubuntu 24.04 – Updated and Networked	2026-09-21 18:11:30Z	Updated Ubuntu, SSH, NAT, and static host-only address; before Wazuh
Wazuh 4.14 Installed and Secured – Pre-Agent	2026-09-21 20:39:43Z	Wazuh installed, dashboard verified, password changed, Filebeat active

Snapshot	UTC Timestamp	State Captured
Wazuh-4.14-Working-Baseline-2026-09-22	2026-09-22 18:05:00Z	Working baseline after agent enrollment, Sysmon integration, and hardening

A snapshot is a recovery point for the VM and depends on the virtual-disk chain. It does not back up the Windows host, the local Splunk installation, or capture files. Restoring a snapshot discards all later VM changes, and a snapshot is not an independent backup. The post-case snapshot is documented in *SOC Investigation #1*.

8. Network Analysis and Validation

8.1 Nmap Installation and PATH Repair

The Nmap 7.991 installer included Nmap Core, Ncat, Nping, Zenmap, Ndiff, and PATH registration. The bundled Npcap option was deselected because Wireshark had already installed Npcap 1.88. The optional registry-based network-performance changes were not selected.

Even with PATH registration selected, `nmap --version` was not recognized. The executable existed at `C:\Program Files (x86)\Nmap\nmap.exe`, so that folder was added to the user PATH and a new terminal was opened, which resolved the problem. Terminals that are already open keep their earlier environment; a successful installation and a successful command lookup are separate checks.

8.2 Host Discovery

```
nmap -sn 192.168.56.10
```

The `-sn` option performs host discovery without a port scan, and Nmap reported one host up. In the saved capture, frame 1666 is an ARP request asking which system has 192.168.56.10, and frame 1668 is the server's reply to the Windows host at 192.168.56.1. Filtering for ARP traffic involving the server returns eight request-and-reply pairs. The full PCAP contains 8,728 packets over 265.936 seconds, including background traffic. [L2, L7]

8.3 Targeted SYN Scan

```
nmap -sS -Pn -p 22,443,1514,1515,55000 --reason 192.168.56.10
```

The `-sS` option selects a SYN scan, `-Pn` skips host discovery for the already-known host, `-p` limits the scan to five ports, and `--reason` reports why each port state was assigned. A SYN/ACK reply from each target port indicated that the port was open. These options are explained in Nmap's official documentation. [L7]

Capture Frame	Source	Destination / Reply	Meaning
1057–1061	Windows, source port 52312	Five selected server ports	Initial SYN probes
1062	Server port 22	Windows 52312	SYN/ACK: SSH port open
1063	Server port 443	Windows 52312	SYN/ACK: dashboard port open

Capture Frame	Source	Destination / Reply	Meaning
1064	Server port 1515	Windows 52312	SYN/ACK: enrollment port open
1065	Server port 1514	Windows 52312	SYN/ACK: agent event port open
1066	Server port 55000	Windows 52312	SYN/ACK: API port open

Table data regenerated from the original PCAP on November 10, 2026. This is an evidence table, not a screenshot of Nmap or Wireshark. The complete capture contains 1,435 packets; a SYN filter isolates the ten scan packets. [L2]

The five SYN/ACK replies are the decisive evidence. No final handshake ACK was sent in these scan exchanges, which distinguishes them from the full TCP connection test in the later investigation. The original record also noted no corresponding reset frames in this capture; that observation applies only to this capture and interface.

8.4 Service-Version Probing

```
nmap -sV --version-light -Pn -p 22,443,1514,1515,55000 --reason 192.168.56.10
```

The `-sV` option adds service and version detection, and `--version-light` limits it to the most likely probes. The first attempt was not captured because Wireshark was still on its welcome screen, so capture was started on Ethernet 2 and the same authorized scan was repeated. The saved file contains 1,255 packets over 34.704 seconds.

Port	Recorded Nmap Result	Supported Assessment
22	OpenSSH 9.6p1 Ubuntu-3ubuntu13.19	SSH banner observed; a version string is not itself a vulnerability finding
443	`ssl/https?`; TLS and web responses	Dashboard listener; the service probes produced real TLS traffic in this capture
1514	`fujitsu-dtcns?`	Nmap's uncertain guess from its port table; the server-side check identifies Wazuh event transport
1515	`ssl/ifor-protocol?`	Uncertain guess; the server-side check identifies Wazuh enrollment
55000	Unknown service	Open listener; the server-side check identifies the Wazuh API

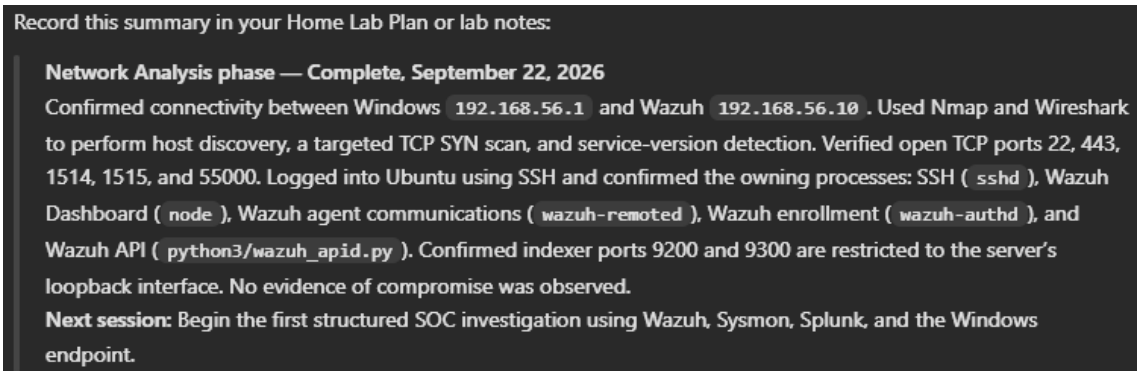
The existing agent connection dominated the capture and continued during probing. The initial probe frames (396–405) were independently rechecked, and one recognized SSH record contains the server's protocol banner. The TLS exchanges in this capture should not be confused with the zero-payload TCP test in the later investigation.

8.5 Validate the Owning Processes Inside Ubuntu

```
sudo ss -lntup
sudo ps -ww -p 713,856,865,1142,1185,1331 -o pid,user,comm,args
```

The `ss -lntup` command lists listening sockets and the processes that own them, and `ps -ww` displays the selected processes' full command lines without truncation. The check identified `sshd` on 22, the Node.js dashboard on 443, `wazuh-remoted` on 1514, `wazuh-authd` on 1515, and `python3 wazuh_apid.py` on 55000. The indexer's Java processes listened on 9200 and 9300 on loopback only. The PIDs applied only to that session and should not be reused as permanent identifiers.

The address `0.0.0.0` is the IPv4 wildcard address (all interfaces), `[::]` is the IPv6 wildcard, and `127.0.0.1` is local loopback. The internal check resolved Nmap's uncertain labels. Together, external observation, packet review, and internal ownership checks verified each service instead of trusting a name inferred from a port number.


Record this summary in your Home Lab Plan or lab notes:

Network Analysis phase — Complete, September 22, 2026
Confirmed connectivity between Windows `192.168.56.1` and Wazuh `192.168.56.10`. Used Nmap and Wireshark to perform host discovery, a targeted TCP SYN scan, and service-version detection. Verified open TCP ports 22, 443, 1514, 1515, and 55000. Logged into Ubuntu using SSH and confirmed the owning processes: SSH (`sshd`), Wazuh Dashboard (`node`), Wazuh agent communications (`wazuh-remoted`), Wazuh enrollment (`wazuh-authd`), and Wazuh API (`python3/wazuh_apid.py`). Confirmed indexer ports 9200 and 9300 are restricted to the server's loopback interface. No evidence of compromise was observed.
Next session: Begin the first structured SOC investigation using Wazuh, Sysmon, Splunk, and the Windows endpoint.

Figure 3. *Network-analysis completion note from September 22, 2026, recording the verified ports, server-side process checks, and readiness for the first investigation. [L4]*

8.6 Useful Display Filters

```
arp
ip.addr == 192.168.56.10 || arp
tcp && ip.addr == 192.168.56.10
ip.addr == 192.168.56.10 && tcp.port in {22, 443, 1514, 1515, 55000}
tls.handshake
http
tcp.flags.reset == 1 || tcp.flags.fin == 1
```

Current Wireshark display-filter syntax requires commas between set members. [L8] TCP stream numbers are specific to each capture file; a stream number from one file should not be applied to another without checking its endpoints.

9. Troubleshooting Record

9.1 Method

Each problem was treated as a small investigation: preserve the exact symptom, find the earliest meaningful error, inspect the relevant layer with read-only checks, make one targeted change, and verify the result. Network reachability, service state, listener state, authentication, and application health were tested separately.

Problems tied to a specific installation step are documented in that step's section. The table below records the remaining problems that do not belong to a single step.

9.2 Other Problems and Fixes

Symptom	Evidence and Diagnosis	Action and Result
Wazuh Script Would Not Run	Filename typo: `wazuh-install-sh`	Downloaded and ran the correct `wazuh-install.sh`
Password Utility Silent	No visible progress did not mean failure	Checked the process from another session and let it finish; the new login worked
VM Boot Stalled at "Loading Essential Drivers"	The VirtualBox log showed the disk and adapters initialized with no related fatal error	After a sustained stall, one VM reset restored boot; no snapshot restore was needed
Error from `ps --ww`	Incorrect long-option form	Corrected to `ps -ww`
Agents Management Not Visible	The dashboard navigation menu was collapsed	Expanded the menu and used *Agents management > Summary*

10. Validation and Readiness

Before the first investigation, each layer of the lab was tested on its own. The table summarizes the final validation results.

Layer	Test	Recorded Result
Endpoint Sensor	Recent `Get-WinEvent` records	Sysmon channel receiving events
Wazuh Integration	Agent status and `whoami` process search	Agent Active; Sysmon process record found
Splunk Integration	Effective input settings and `whoami` search	XML input accepted; test process found in `main`
Network	ARP discovery and five-port SYN scan	Host up; five expected services reachable
Service Identity	Nmap plus `ss` and `ps`	Owning processes confirmed; indexer bound to loopback only
Administration	SSH port check and login	Working management session
Recovery	Three named pre-case snapshots	Known rollback points retained

11. Lessons Learned and Future Plans

11.1 Lessons Learned

- **Verify the File, Not the Download.** A completed transfer proves nothing; a checksum match against the publisher's value does (Section 3.1).
- **Confirm That a Change Was Saved Before Applying It.** Printing a configuration file before applying it exposes edits that never reached the disk (Section 4.3).

- **Test Each Layer Separately.** Reachability, service state, ingestion, and collection each need their own check; success at one layer does not prove the next (Sections 4.4 and 5.4).
- **Give a New Platform Time to Initialize.** Heavy load right after installation can look like failure. Waiting and performing one orderly restart preserves the evidence that repeated restarts would erase (Section 5.2).
- **Check Coverage Before Trusting an Empty Result.** An empty search can reflect missing inputs or excluded indexes rather than a fault (Section 3.4).
- **Confirm the Recorder Is Running Before Generating Evidence.** Starting the capture first and confirming that packets arrive avoids repeating a test (Section 8.4).
- **Identify Services from Inside the Host.** Port-based guesses from an external scan need confirmation from listener and process checks on the server (Section 8.5).
- **Keep Credentials Out of Evidence.** Screenshots should be reviewed for secrets before they are saved or shared (Section 5.2).
- **Snapshot Before Each Major Change.** Named snapshots at each stage provide clean rollback points (Section 7.3).

11.2 Accepted Trade-Offs

Several configuration choices favor simplicity over the hardening a production environment would require. Each was accepted deliberately for a single-owner home lab.

Trade-Off	Why It Is Acceptable for This Lab	What Would Change It
UFW Inactive on the Ubuntu VM	The server is reachable only from the Windows host over the host-only link. VirtualBox NAT does not accept inbound connections unless port forwarding is configured, and none is recorded in the VM definition.	Bridged networking or additional lab hosts. UFW would then allow only ports 22, 443, 1514, 1515, and 55000 from 192.168.56.0/24.
SSH Password Authentication	SSH is reachable only from the owner's workstation over the host-only link, and the server has a single administrator account.	Remote access or additional users. Key-based authentication would replace password login.
Unencrypted VM Disk	The virtual disk resides on a home desktop with no meaningful physical-theft risk and holds lab test data, consistent with the decision not to enable BitLocker on the host.	Storing sensitive data in the lab or moving the VM to a portable device.
Personal Workstation as the Monitored Endpoint	A real workstation produces realistic telemetry and kept the initial setup simple. Active scanning targeted only the lab VM.	Event noise or privacy concerns from personal activity. A dedicated Windows VM is planned (Section 11.3).
Outbound Internet Access Through NAT	The server needs package and security updates, so the lab is intentionally not air-gapped.	Testing untrusted samples or malware, which would require an isolated network.

11.3 Remaining Plans

The following items were planned or discussed but not completed before the first investigation. They are future work, not installed capabilities.

Item	Plan
Dedicated Windows VM	A separate monitored Windows endpoint to isolate lab telemetry from the personal desktop.
Additional Lab VMs	Further Windows and Linux systems to broaden the monitored environment.
Detection Tuning	Tune broad Sysmon collection and repeated shell alerts based on reviewed evidence, rather than suppressing whole categories of PowerShell, Temp-folder, or network events.
Response Automation	Start with Wazuh's built-in capabilities before evaluating a dedicated SOAR platform.
Ticketing Integration	Connect alerts to a case or ticket workflow.
Threat Intelligence and Guided Practice	Threat-intelligence references and access to guided SOC practice platforms.

12. Conclusion and Handoff

This report documents the configuration of the home lab: planning the environment, installing and connecting each tool, troubleshooting the problems that arose, and validating that the lab could produce and find evidence. It does not cover an investigation.

The network-analysis phase was recorded as complete on September 22, 2026. With every layer validated (Section 10), the lab was ready for its first structured investigation, which would have its own authorization, hypotheses, test timing, baseline, evidence, and response decision.

Appendix A. Evidence Integrity and Provenance

The three setup captures were read offline during report preparation; no new scans or VM changes were made. Each complete-file hash covers all captured background traffic, not only the filtered evidence.

Capture	Size / Packets	SHA-256 (Rechecked November 10, 2026)
01-nmap-host-discovery.pcapng	4,248,728 bytes; 8,728 packets	`7B1AF7B45A96EA7D26020C9742E5FF7E9F19D10D969829B838CF710DA00E4B01`
02-wazuh-tcp-syn-scan.pcapng	808,420 bytes; 1,435 packets	`4C798A03688AEE094A161AF9D478D03EFAEF08E606ADE7BD3A4135F089A86550`
03-wazuh-service-version-scan.pcapng	548,368 bytes; 1,255 packets	`EA5C586F05FB4CC04FD0B144DF081DB78C0BFF3A938ED89430FF41DCDC0D5434`

The evidence now resides in the project folder; the September notes reference an earlier Wireshark folder. The current location and the original acquisition location are distinct provenance facts, and a matching file hash is stronger evidence of byte-for-byte identity than a matching filename. For publication, the host name and Linux administrator name are replaced with the generic labels WORKSTATION-01 and labadmin; all other recorded values are unchanged.

Figure 3 reproduces the network-analysis note. The architecture diagram and packet tables identify the configuration records or capture frames used to prepare them. Historical application observations are attributed to the original lab record.

Appendix B. Sources and References

[L1] Saved project files: `install-cyber-lab-foundation.ps1`, `sysmon-lab-config.xml`, `READ-ME-FIRST.md`, and `download_ubuntu_iso.py`. The script's WSL option reflects early planning; the deployed lab uses VirtualBox.

[L2] Three original PCAPNG files dated September 22, 2026; read with TShark and Capinfos, with SHA-256 values rechecked November 10, 2026.

[L3] Saved `Wazuh-Server.vbox` definition, read November 10, 2026. Confirms VM resources, NAT and host-only adapters, and named snapshots; it does not establish current Linux service health.

[L4] *Home Lab – Network Analysis.png*, phase summary dated September 22, 2026 (Figure 3).

[L5] Wazuh documentation, consulted November 10, 2026: Quickstart · Architecture. Recorded 4.14 commands are kept as the versions used at setup.

[L6] Microsoft Sysmon: Sysmon download and overview · Sysmon event reference

[L7] Nmap: Options summary · Port-scanning techniques

[L8] Wireshark: Display-filter expressions, including set-membership syntax.

[L9] NIST SP 800-61 Rev. 3. Lab preparation supports the Govern, Identify, and Protect functions; the later case exercises Detect, Respond, and Recover. This mapping is not a certification or compliance claim.

[L10] *PC Build*, Jackson Blase Home Lab report series, Sections 1.3 and 4.2. Documents the host's Ryzen 7 9700X, 64 GB of memory, and the SVM virtualization setting used by VirtualBox.

[L11] Additional references for the glossary: VirtualBox 7.2 User Guide · Splunk default fields · Microsoft Defender Antivirus compatibility and modes

Glossary

Terms are defined as they are used in this report. Authoritative references are listed in Appendix B.

Agent and Enrollment	An agent collects endpoint telemetry and sends it to the Wazuh manager. Enrollment registers the agent with the manager and issues the key used to authenticate its communication.
Air Gap	Physical or logical isolation that leaves a system with no connection to outside networks. This lab is not air-gapped because the server uses NAT for updates.
Alert and Telemetry	Telemetry is recorded system activity. An alert is a rule-generated record that flags selected activity for review.
API and Wazuh Server API	An application programming interface lets software make structured requests to other software. The Wazuh server API provides management and status functions, including agent operations.
ARP	Address Resolution Protocol maps an IPv4 address to a hardware (MAC) address on the local network segment. ARP replies can reveal live hosts during local discovery.
Authenticode and SHA-256/ SHA-512	Authenticode is Microsoft's code-signing technology; checking a signature verifies the publisher and confirms the file has not changed since signing. SHA-256 and SHA-512 produce hash values that can be compared with trusted published values.
Capture Filter and Display Filter	A capture filter controls which packets are recorded. A display filter controls which recorded packets are shown.
Credential Rotation	Replacing a password or key and updating every account or service that depends on it.
Defender Passive Mode	A Microsoft Defender Antivirus mode in which another product provides primary real-time protection while Defender stays installed and can still be updated.
DHCP, Static IP, and CIDR	DHCP assigns network settings automatically. A static IP address is set manually. CIDR notation uses a prefix length, such as /24, to mark how many bits identify the network.
Endpoint and Hostname	An endpoint is a monitored device. A hostname identifies a system on the network; Wazuh can register the same system under a separate agent name.
Event Channel, EVTX, and XML	A channel is a named Windows event log stream. EVTX is the on-disk event-log file format. XML is a structured text representation of event data.
Filebeat	A lightweight log shipper that forwards Wazuh manager alerts to the indexer for storage and search.
Host Discovery	Determining which devices are reachable on an authorized network before examining individual ports or services.
Host-Only Networking	A VirtualBox network that connects the host and its guests over a private virtual link with no external access. The lab uses it for monitoring and server administration.
Hypervisor, Host, and Guest	A hypervisor runs virtual machines. The host provides the physical resources, and the guest is the operating system running inside a VM.
ISO and LTS	An ISO is a disk-image file, used here as installation media. LTS (long-term support) denotes an Ubuntu release with an extended maintenance period, normally five years of standard support.

Kernel-Mode Stack Protection	A Windows feature that uses CPU shadow stacks to detect tampering with kernel return addresses, blocking return-oriented programming attacks.
Key-Based SSH Authentication	SSH sign-in using a cryptographic key pair instead of a password. The server holds the public key, and only the holder of the matching private key can sign in.
Listening Socket and Port	A listening socket waits for incoming connections at an address and port. A port number identifies a transport-layer endpoint on a host.
Load Average	A Linux measure of the number of tasks that are running, waiting to run, or in uninterruptible sleep, averaged over 1, 5, and 15 minutes. Values well above the CPU count indicate system pressure.
Loopback and Wildcard Bind	Loopback addresses such as 127.0.0.1 are reachable only from the same host. A wildcard bind such as 0.0.0.0 listens on all local IPv4 interfaces.
LSA Protection	A Windows setting that runs the Local Security Authority process as a protected process, blocking unauthorized code from reading credentials or injecting into it.
LVM and LUKS	Logical Volume Manager pools disk space into resizable logical volumes. Linux Unified Key Setup is the standard Linux disk-encryption format.
Memory Integrity	Also called hypervisor-protected code integrity (HVCI). It uses virtualization-based security to block unsigned or tampered code from running in the Windows kernel.
NAT	Network address translation. VirtualBox's NAT mode routes guest traffic through the host's network connection; the lab used it for outbound updates and downloads.
Netplan and YAML	Netplan applies Ubuntu network settings from configuration files written in YAML, an indentation-based structured text format.
Nmap, SYN Scan, and Service Probes	Nmap discovers hosts and ports. A SYN scan infers a TCP port's state from the response to a SYN packet without completing the handshake. Service probes use protocol exchanges or banners to identify software.
Npcap, Wireshark, and TShark	Npcap provides packet-capture access on Windows. Wireshark is the graphical packet analyzer, and TShark is its command-line counterpart.
PATH	The environment variable that lists the directories a shell searches for executable commands.
PCAPNG	PCAP Next Generation, a capture file format that stores packets along with interface and capture metadata.
Port Forwarding	A NAT rule that passes inbound connections on a host port to a port on the guest. Without one, VirtualBox NAT accepts no inbound connections to the VM.
SIEM	Security information and event management. A SIEM centralizes, correlates, and searches security records to support monitoring and investigations.
Snapshot and Backup	A snapshot records a VM's state for rollback and depends on the original virtual disk. A backup is a separate copy that remains usable if the original VM files are lost.
Socket Activation and `systemd`	The `systemd` service manager controls services on Ubuntu. With socket activation, `systemd` listens on a port itself and starts the corresponding service only when a connection arrives.
SPL, `_raw`, and `_time`	Splunk's Search Processing Language filters and transforms events. The `_raw` field holds the original event data, and `_time` holds the timestamp Splunk assigned to the event.

Splunk Index, Source, and Sourcetype	An index stores searchable events. Source identifies where an event came from, such as a Windows channel. Sourcetype identifies the event's format and how Splunk should parse it.
Splunk `inputs.conf` and `btool`	The `inputs.conf` file defines what data Splunk collects. The `btool` utility shows the effective merged configuration and which file supplied each setting.
SSH and Host-Key Fingerprint	Secure Shell provides encrypted remote administration. A host-key fingerprint is a short digest of the server's public key, used to confirm the server's identity on first connection.
Sysmon and Event ID	Microsoft System Monitor records activity such as process starts, network connections, and file creation. Each event ID identifies a type of activity; for example, ID 1 is process creation and ID 3 is a network connection.
TCP, TLS, and HTTPS	TCP provides a reliable transport connection. TLS encrypts and authenticates application traffic over that connection. HTTPS is HTTP carried over TLS.
UFW	Uncomplicated Firewall, Ubuntu's front end for managing host firewall rules.
VDI and vCPU	VirtualBox Disk Image is the VM's virtual-disk file format. A vCPU is a virtual processor assigned to the guest.
Wazuh Manager, Indexer, and Dashboard	The manager decodes events and applies rules. The indexer stores and searches alert records. The dashboard is the web interface for investigation and administration.
WSL	Windows Subsystem for Linux, a Windows feature for running a Linux environment. The lab's Ubuntu server instead runs as a VirtualBox VM.